Prediction of Bank Term Deposit Subscription Using Data Mining Classification Algorithms

Abdalkarim M. S. Alaraj
Dept. of Artificial Intelligence and Data Engineering
International University of Sarajevo
aalaraj@student.ius.edu.ba

Muhamed Maglić
Dept. of Artificial Intelligence and Data Engineering
International University of Sarajevo
mmaglic@student.ius.edu.ba

Hamza Hubanić
Dept. of Artificial Intelligence and Data Engineering
International University of Sarajevo
hhubanic@student.ius.edu.ba

Faculty of Engineering and Natural Sciences (FENS) — Supervisor: Prof. Emine Yaman (eyaman@ius.edu.ba)

***Abstract*— Banks depend a lot on direct marketing campaigns, usually phone calls, to pitch financial products like term deposits. Since calling every single customer is expensive and kind of slow, it helps to guess ahead of time which clients might subscribe. In this project we look at the Bank Marketing dataset from the UCI Machine Learning Repository, which has roughly 45,000 clients from a Portuguese bank and 16 attributes. We treat it like a binary classification problem, then we compare five models that we studied in our Data Mining class: a Decision Tree, Naive Bayes, k-Nearest Neighbors, Random Forest, and Logistic Regression. Before fitting anything, we ran a full preprocessing pipeline that basically encoded the target, dropped the duration attribute that leaked information, handled the pdays sentinel value, dealt with missing values, removed duplicates, performed one hot encoding, and finally scaled the features. For evaluation we used accuracy, precision, recall, F1-score, and ROC-AUC. The outcomes suggest all models get a solid accuracy near 89%, but that number can be a bit deceptive, because the dataset is imbalanced. If we focus on F1-score and recall, Naive Bayes seems strongest at catching the actual subscribers, while Random Forest leads on ROC-AUC, so it tends to rank the most promising clients at the top. We also ran K-Means clustering, plus a Random Forest feature-importance analysis, just to interpret the data a little more clearly. Finally, we tested class weighting to handle the imbalance, which strongly increased the recall of the Decision Tree and Logistic Regression models (Logistic Regression went from finding 18% to 60% of the subscribers) at the cost of lower precision. The main conclusion is that overall accuracy alone is not enough on imbalanced data, and the choice of model depends on the bank's goal.**

Keywords*— *data mining; classification; bank marketing; term deposit; imbalanced data; Random Forest; Naive Bayes; ROC-AUC.

I. INTRODUCTION

Marketing is one of the main ways that banks attract new customers and sell their products. A very common product is the term deposit, where a client agrees to keep a fixed amount of money in the bank for a certain period in exchange for an interest rate. To sell these deposits, banks run direct marketing campaigns, and one of the most used methods is calling clients on the phone [1]. However, calling clients is expensive and takes a lot of time, and most of the clients that are contacted do not end up subscribing. Because of this, it would be very useful for a bank to know in advance which clients are more likely to say yes, so that the marketing team can focus its effort on them.

This kind of problem is a good fit for data mining. Data mining is the process of finding useful patterns and some sort of knowledge from huge volumes of data [2]. One of the main tasks in data mining is classification, where the aim is to put each record into one of a fixed set of classes [3]. In our situation the two classes are straightforward: the client either signs up for the term deposit (yes) or does not (no). This classification is a supervised task, meaning the model learns from earlier examples where the correct class is already known, and then it predicts the class of brand-new cases. This really fits the bank context, because the bank already keeps records from prior campaigns, so it knows who subscribed and who did not. In this project we rely on the Bank Marketing dataset, which was gathered by a Portuguese bank and is publicly shared through the UCI Machine Learning Repository [4]. The dataset contains details about the clients, the current marketing campaign, plus a summary of previous campaigns. Our objective is to construct and compare a few classification models that estimate whether a client will subscribe to a term deposit, as well as to figure out which client's traits matter the most. We picked five classification approaches that were covered during our Introduction to Data Mining course: Decision Tree, Naive Bayes, k-Nearest Neighbors (k-NN), Random Forest, and Logistic Regression. We chose these five because they come from different families of methods (tree-based, probabilistic, distance-based, ensemble, and linear), so comparing them is more interesting than comparing very similar models. In addition to classification, we also applied K-Means clustering to group similar clients, and we used the Random Forest model to find the most important features.

The rest of this paper is organized as follows. Section II reviews related work. Section III describes the dataset, the preprocessing steps, the five algorithms, and the evaluation metrics. Section IV presents the results and discusses them, including the exploratory analysis, the model comparison, the feature importance, and the clustering. Section V gives the conclusion, and Section VI lists possible future work.

II. RELATED WORK

The Bank Marketing dataset has been studied by several researchers. The dataset itself was introduced by Moro, Cortez, and Rita [4], who used a data-driven approach to predict the success of bank telemarketing. In an earlier work, Moro, Laureano, and Cortez [5] applied the CRISP-DM methodology to the same kind of bank marketing data and showed that data mining can support marketing decisions. Elsalamony [6] analyzed the same dataset using several data mining techniques, including neural networks and decision trees, and reported that handling the imbalance of the classes is an important issue for this problem.

Comparing several classification algorithms on a single dataset is a common thing in both student's and research projects. For instance, in the medical domain, several studies have compared Naive Bayes, Logistic Regression, Random Forest, and a bunch of related approaches to forecast heart disease [7], and they usually conclude that no one algorithm is always the best, period. This whole "compare classifiers" idea shows up in other application zones too, for example customer satisfaction prediction. In general, these works suggest that the quality of an algorithm's performance depends heavily on the dataset, and how its data is prepared.

One theme that comes back a lot is class imbalance. In direct marketing, the number of clients who accept an offer is nearly always much smaller than the who refuse, so the dataset is basically biased toward the negative class. He and Garcia [22] point out that standard classifiers trained on imbalanced data often lean toward the majority class, giving high accuracy, but a weaker precision. To address this, different techniques have been proposed. A well-known one is SMOTE, the oversampling method introduced by Chawla et al. [24], which generates new synthetic samples for the minority class. Also, a lot of studies involving the Bank Marketing dataset describe the same snag and emphasize that evaluation metrics beyond accuracy are needed.

Our work basically stays in the same broad spirit of comparing classifiers, but it concentrates on the bank marketing problem and pays close attention to the fact that the dataset is imbalanced. Rather than only giving an accuracy number, we provide precision, recall, F1-score, and ROC-AUC at the same time, and we also examine how the ordering of the models shifts depending on the chosen metric. Additionally, we blend the classification analysis with clustering plus feature-importance analysis.

III. MATERIALS AND METHODS

A. Dataset Description

The Bank Marketing dataset has around 45,211 records and 16 input attributes, plus one target attribute. Each record is basically one client who was reached during some marketing campaign and yep that's it. In total, those 16 attributes can be sort of grouped into three categories: details about the client, facts about the last touchpoint of the current campaign, and data tied to the previous campaigns. Table I shows all the attributes and what they mean.

TABLE I. Description of the Dataset Attributes

Attribute	Type	Description
age	numeric	Client age in years
job	categorical	Type of job (admin, technician, retired, ...)
marital	categorical	Marital status
education	categorical	Education level
default	categorical	Has credit in default?
balance	numeric	Average yearly balance (euros)
housing	categorical	Has a housing loan?
loan	categorical	Has a personal loan?
contact	categorical	Contact communication type
day	numeric	Last contact day of the month
month	categorical	Last contact month
duration	numeric	Last contact duration (seconds)
campaign	numeric	Number of contacts in this campaign
pdays	numeric	Days since last contact (-1 = never)
previous	numeric	Number of contacts before this campaign
poutcome	categorical	Outcome of the previous campaign
y (target)	categorical	Subscribed to term deposit? (yes / no)

The target attribute y is the class we want to predict. As is shown later in Fig. 1, the dataset looks strongly imbalanced: only around 11.7% of the clients subscribed while the large majority did not. That imbalance is rather important, and it changes how we should interpret the results.

B. Data Preprocessing

Real datasets are rarely clean, so preprocessing is an essential step before training any model [2]. We applied the following steps in order.

1) Target encoding: We mapped the target values yes to 1 and no to 0, so it becomes a binary variable that the models and metrics can work with.

2) Removing the duration attribute: The duration attribute is the length of the last phone call in seconds. Unfortunately, this number is only known after the call happened already, so you cannot use it to decide who to contact beforehand. The scores could become unrealistically high, if we kept it.

3) Handling the pdays sentinel: In the pdays attribute, the value -1 does not actually mean a real count of days. It means the client was never contacted in a previous campaign. So, we replaced -1 with 0, and we also created a new binary variable, was_contacted_before, to retain that piece of information rather than discarding it.

4) Missing values: A few attributes (like job, education, contact, and poutcome) include missing entries. The data loader stores these as empty values, NaN, mainly for clients who were not part of a previous campaign. We did not delete those rows. Later, when the one-hot encoding is applied, a missing category simply ends up as 0 in all dummy columns for that attribute. So, no client disappears, and the absence itself is treated automatically.

5) Duplicate removal: We got rid of the exact duplicate records, so it does not show up more than once.

6) Outliers: We inspected the numeric attributes such as balance and campaign with boxplots. Many of the extreme values are real clients (for example, a client with a very high balance), not errors, so we decided not to remove them. The scaling step below reduces their effect.

7) One-hot encoding: The categorical attributes were converted into numeric form using one-hot encoding,

because algorithms such as Logistic Regression, k-NN, and Naive Bayes need numeric input. After encoding, the number of features increased.

8) Train/test split: We split the data into 75% for training and 25% for testing. We used stratified sampling so that both parts keep the same proportion of subscribers as the full dataset.

9) Feature scaling: We standardized the numeric features so that they all have a similar range. This is important for k-NN and Logistic Regression, which are sensitive to the scale of the values. Decision Tree and Random Forest do not need scaling, but it does not harm them.

C. Classification Algorithms

We used five classification algorithms. Each one comes from a different family, which makes the comparison more meaningful.

1) Decision Tree: A decision tree splits the data step by step using simple rules on the attributes, building a tree where each leaf gives a class [8], [9]. Trees are easy to understand and explain, which is useful for non-technical staff. Their main weakness is that if they grow too deep, they can overfit the training data [3]. We decided to limit the maximum depth of the tree, a bit more than usual, in order to have kind of control over this.

2) Naive Bayes: A probabilistic classifier grounded in Bayes' theorem [10]. It basically takes the attributes as if they are independent of one another given the class label, which is often not actually the case, but somehow it still behaves well in real usage, and it is also fast [11]. We applied the Gaussian form, since it fits numeric attributes nicely.

3) k-Nearest Neighbors (k-NN): Classifies a new client by looking at the k most similar clients (its nearest neighbors) and taking a majority vote [12]. It does not build a model during training; instead, it stores the data and computes distances when predicting. Because it relies on distances, scaling the features is very important [3]. We used k = 15.

4) Random Forest: An ensemble technique that builds many decision trees using random slices of the dataset and then mixes their votes [13], [14]. If you take the average over all those trees, it ends up with more reliable outcomes than just relying on one lone tree, also tending to be less prone to overfitting. Additionally, it can also indicate the most important feature. The downside is that it runs slower and it is a bit harder to interpret compared to a single tree.

5) Logistic Regression: Is basically a linear model for binary classification [15]. It figures out the likelihood that a client ends up in the positive class, and the coefficients basically tell you how each feature plays a role in the prediction. It is straightforward, it runs quickly, and it is easy to read, but because it only captures linear decision boundaries, it can miss those more intricate patterns that show up in real data.

D. Clustering Method

In addition to classification, we used K-Means clustering [16], [17] to explore the data without using the target. K-Means groups the clients into k clusters so that clients in the same cluster are similar to each other. We used the elbow method to choose the number of clusters and then projected the data into two dimensions using Principal Component Analysis (PCA) [18] so that the clusters could be visualized.

E. Evaluation Metrics

Because the dataset is imbalanced, accuracy alone is not enough to judge the models. A model that always predicts "no" would already reach about 88% accuracy without learning anything useful. For this reason, we also used precision, recall, F1-score, and ROC-AUC [19], [20]. These metrics are computed from the confusion matrix, which counts the true positives, false positives, true negatives, and false negatives. Precision measures how many of the clients predicted as subscribers really subscribed. Recall, also called the true positive rate, measures how many of the real subscribers the model managed to find. There is usually some tradeoff between the two making a model predict "yes" more often increases recall, but it also lowers precision. The F1-score, that kind of thing, is the harmonic mean of precision and recall, and it gives one balanced number, like sort of [21]. ROC-AUC measures how well the model separates the two classes across all possible thresholds, and it is known to be useful for imbalanced data [22], [23]. We also recorded the training time of each model, since training speed can matter when a model has to be retrained often.

IV. RESULTS AND DISCUSSION

A. Exploratory Data Analysis

Before training the models, we explored the data with several plots. Fig. 1 shows the distribution of the target. It is clearly imbalanced: the number of clients who did not subscribe is much larger than the number who did.

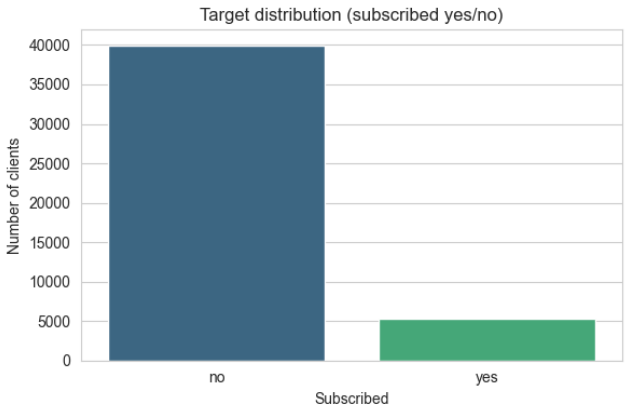

Fig. 1. Distribution of the target variable (subscribed yes/no).

Fig. 2 shows the correlation between the numeric features. Most correlations are weak, which means the features carry different information and there is little redundancy. The strongest correlations are between pdays, previous, and the engineered was_contacted_before feature, which all describe the history of previous contacts. We can also see that no single numeric feature has a strong linear correlation with the target y (the highest is only about 0.17), which already suggests that the prediction will not be easy.

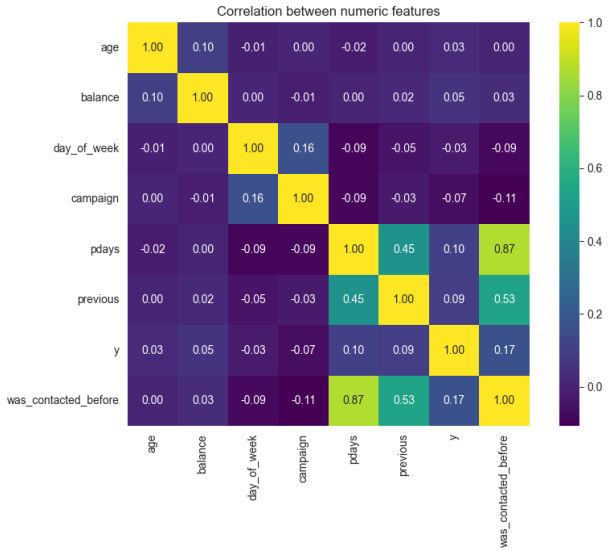

Fig. 2. Correlation heatmap of the numeric features.

Fig. 3 shows the subscription rate by the outcome of the previous campaign (poutcome). Clients whose previous campaign was a success subscribed again at a very high rate, about 64%, which is far higher than for clients whose previous outcome was "failure" or "other" (around 12–16%). This makes sense: a client who accepted an offer before is more likely to accept again.

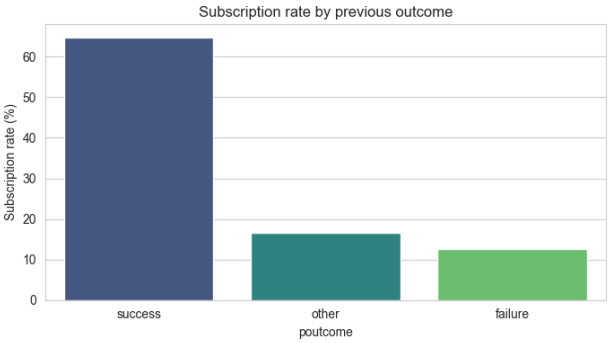

Fig. 3. Subscription rate by previous campaign outcome.

We also observed the distributions of the numeric features, kind of broad. The age of the clients is pretty much bell-shaped, and it sits around the working-age range, but balance, campaign, and previous are strongly right-skewed, like noticeably. Most clients end up with a modest balance and only a few contacts, yet there are a handful of cases with extremely large values. This skew is basically why we standardized the numeric features. And it's also why we didn't delete extreme values, because they are still real clients. When we compared subscribers versus non-subscribers, the subscribers usually had a slightly higher balance. Meanwhile, clients who were contacted many times in the current campaign were less likely to subscribe, so repeatedly calling a reluctant person has little or no benefit.

B. Classification Results

Table II shows the performance of the five models on the test set. All models reach a similar accuracy of about 89%, except Naive Bayes, which is a bit lower at 85.6%. However, as explained before, accuracy is misleading on this imbalanced dataset, so we focus on the other metrics.

TABLE II. Performance of the Five Classifiers

Model	Acc	Prec	Rec	F1	AUC
Naive Bayes	0.856	0.394	0.424	0.409	0.736
Decision Tree	0.893	0.608	0.228	0.331	0.655
Random Forest	0.894	0.627	0.225	0.331	0.771
k-NN	0.894	0.633	0.215	0.321	0.729
Logistic Reg.	0.894	0.670	0.181	0.285	0.754

What is interesting about results is that none of our models wins by all metrics. Naive Bayes has the lowest accuracy, but it also has the highest recall (0.424) and the highest F1-score (0.409). So basically, that suggests Naive Bayes is best at finding the clients who will subscribe, even if it also triggers more false alarms, every now and then. The other four models kind of act in the opposite way: they show strong accuracy and high precision, but their recall stays very low, roughly between 0.18 and 0.23. In other words, they miss most of the real subscribers. They still look good on accuracy mainly because they predict "no" repeatedly, which is exactly the usual imbalanced-data trap.

Logistic Regression has the highest precision (0.670) but the lowest recall (0.181). That means when it predicts "yes" it is likely correct, however it says "yes" far too rarely. Decision Tree, Random Forest, and k-NN are in the middle, with fairly similar F1-scores around 0.32–0.33.

Fig. 4 shows the ROC curves for all five models. Here it is a bit clearer: Random Forest has the highest ROC-AUC of 0.771, then Logistic Regression 0.754 and after that Naive Bayes 0.736. The Decision Tree has the lowest ROC-AUC (0.655). A higher ROC-AUC means that, if we used the model's predicted probability to rank clients from most to least likely to subscribe, Random Forest would give the best ranking. This is very useful in practice, because a bank could simply call the top clients on the ranked list first.

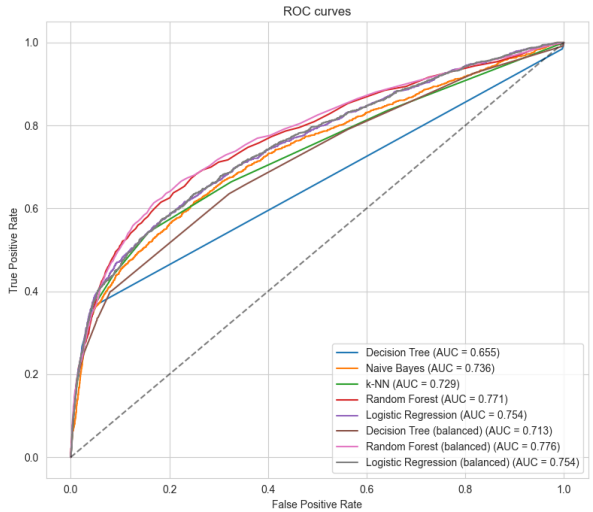

Fig. 4. ROC curves of all models, including the class-weighted versions.

Putting these results together, the “best” model depends on the goal. If the bank wants to catch as many potential subscribers as possible at the default threshold, Naive Bayes is the most useful because of its higher recall and F1. If the bank wants to rank clients and call the most promising ones first, Random Forest is the best because of its higher ROC-AUC. The low recall of most models also shows that the imbalance should be handled directly, which we discuss in the future work.

Regarding training time, k-NN was the fastest to train (0.01 s) because it does not build a model during training, while Random Forest was the slowest (1.19 s) because it builds many trees. However, speed is not a concern at this dataset size, since all training times are very small.

It is also worth comparing our results with the related work and whatever is similar. Studies that use the same dataset often report accuracy values in a comparable interval to ours, like 88–90% [6], so this sort of a confirmation that our pipeline behaves as intended. The recall being low matches what He and Garcia [22] already discussed about the imbalance issue. And because of that, it becomes kind of the main reason why just reporting accuracy, like some earlier studies already did, can leave a too optimistic impression about how the model really works in practice. That is why we think that the mix of metrics we used gives a fairer assessment.

C. Summary of the Comparison

To make the comparison easier to read, Table III basically summarizes which model lands best on each metric. The table kind of makes the main message of this study very clear: the answer to “which model is best?” is not just one single name, but it depends on what the bank actually cares about. If the goal is to grab as many subscribers as possible, the recall and F1 columns point toward Naive Bayes. If the goal is more about not wasting calls on clients who won’t end up subscribing, then the precision column points to Logistic Regression. And if the goal is to arrange clients for a calling list, the ROC-AUC column indicates Random Forest.

TABLE III. Best Model per Evaluation Metric

Metric	Best Model
Accuracy	Logistic Regression (0.894)
Precision	Logistic Regression (0.670)
Recall	Naive Bayes (0.424)
F1-score	Naive Bayes (0.409)
ROC-AUC	Random Forest (0.771)
Training speed	k-NN (0.01 s)

D. Feature Importance

Fig. 5 shows the 15 most important features from the Random Forest model. The top features are balance, age, and day, followed by campaign and the previous-outcome-success feature. This somewhat matches the exploratory analysis; the success of the previous campaign ranks among the top features, which aligns with Fig. 3. Meanwhile, the numeric features balance and age rank very high, indicating that the client's financial situation and age are significant factors. The lower rankings of features like housing and education suggest that the client's personal and financial profile is more important than the specifics of a single call.

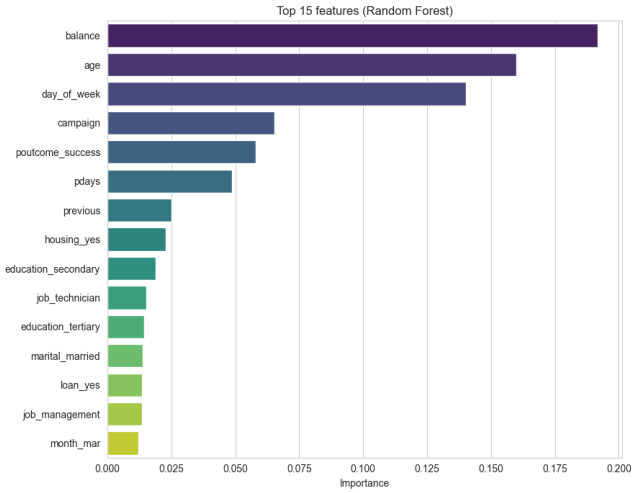

Fig. 5. Top 15 features by Random Forest importance.

E. Clustering Results

Finally, we applied K-Means clustering with four clusters on the numeric features. We projected the result into two dimensions using PCA. Fig. 6 shows the clusters. Most clients are part of a large, dense group, while smaller clusters include clients with more extreme values, such as very high balance or many previous contacts. Since the clustering considered features like balance and contact history, the separation we observe mainly comes from these attributes. The clustering is exploratory and does not incorporate the target. However, it supports the same conclusion from the classification and feature-importance analysis: financial situation and contact history are the main factors that distinguish different types of clients.

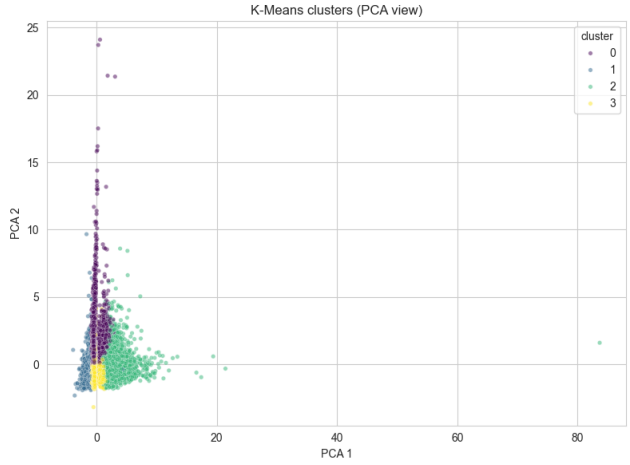

Fig. 6. K-Means clusters shown in the PCA projection.

From a practical point of view, these results kind a have clear meaning for the bank. Because the models can sort clients by their predicted probability of subscribing, the bank does not have to treat the prediction as a strict yes/no

decision. Instead, it can use the ranking to build a calling list, starting with the clients who have the highest predicted probability. Even if the recall at the default threshold is low, a good ranking (high ROC-AUC) still helps the marketing team to reach a large share of the future subscribers while calling only part of all clients. This is basically the cost saving thing that motivated the project in the first place, and it also indicates that the usefulness of a model depends on how it is used, not only on one single accuracy number.

F. Effect of Class Weighting

The low recall of the original models kind of show that they miss most of the real subscribers, pretty much. So, to address that we retrained the three models that support it, namely Decision Tree, Random Forest, and Logistic Regression with the option class_weight = "balanced", this makes the minority class count matter more during training. Naive Bayes and k-NN do not give this option, so they were left as they are, unchanged, or in other words. Table IV compares the original versus the weighted versions.

TABLE IV. Original vs. Class-Weighted Models

Model	Acc	Prec	Rec	F1	AUC
Decision Tree	0.893	0.608	0.228	0.331	0.655
Decision Tree (bal.)	0.856	0.389	0.402	0.395	0.713
Random Forest	0.894	0.627	0.225	0.331	0.771
Random Forest (bal.)	0.893	0.644	0.198	0.303	0.776
Logistic Reg.	0.894	0.670	0.181	0.285	0.754
Logistic Reg. (bal.)	0.757	0.264	0.603	0.368	0.754

The effect of class weighting is pretty clear, and it sort of lines up with the theory. For Logistic Regression, the recall jumps from 0.181 to 0.603, so the model now finds around 60% of the real subscribers instead of only 18%… but the cost is that its precision falls from 0.670 to 0.264, and its accuracy drops to 0.757. This happens because it predicts "yes" a lot more often, even if that means more false alarms. The Decision Tree also improves, kind of across the board: its recall goes from 0.228 to 0.402, its F1 rises from 0.331 to 0.395, and its ROC-AUC moves from 0.655 to 0.713. The Random Forest, meanwhile, doesn't really budge much, and its F1 even drops a bit which kind of suggests that class weighting is not some universal win and it depends on the specific model and the dataset.

Two more things are worth pointing out. First, even after applying weight to the other models, Naive Bayes still comes out with the best F1 (0.409) among all eight, so it kind of confirms that it's naturally a good fit for this imbalanced situation. Second, the ROC-AUC numbers hardly change with weighting, which is what you'd expect: weighting mostly tweaks how the model behaves around the decision threshold, not its real knack to rank clients. Overall, class weighting is a straightforward and pretty effective lever when the goal is to detect more subscribers, assuming the bank can tolerate the lower precision that comes along with it.

G. Limitations

Our Analysis has a few limitations, and it is clear. First, we used the default threshold as 0.5 for all the models, and we didn't really tune it at all. That is one reason the recall for the unweighted models is a bit low. Second, we did emphasize class weighting, but we didn't really pit it against other imbalance strategies like oversampling, which we should have considered more. Third, we relied mainly on the default hyperparameters and only adjusted things like the tree depth and the value of k, so, the models could probably be better with proper tuning and a more careful search. Finally, the dataset comes from one bank only, in one country. Consequently, although the results look decent, they might not carry over well to other banks or different regions.

V. CONCLUSION

In this project, we used five classification algorithms on the Bank Marketing dataset to predict whether a client would subscribe to a term deposit, and we also tested class weighting to handle the imbalance. We applied a complete preprocessing pipeline and compared the models using several metrics. What is mainly noticeable on an imbalanced dataset, a high accuracy can be misleading: although four of the models reached about 89% accuracy, most of them missed the majority of the real subscribers. When we looked at recall and F1-score, Naive Bayes had the best findings, while Random Forest achieved the best ROC-AUC and would be the best choice for ranking clients. Adding the class weight attribute to the Decision Tree and Logistic Regression increased their recall strongly, with Logistic Regression going from finding 18% to 60% of the subscribers, at the cost of lower precision. The feature-importance and clustering analyses both similarly pointed to the client's balance, age, and contact history as the most important factors. Overall, this project showed how important preprocessing, the handling of class imbalance, and the right choice of evaluation metric are in a real data mining analysis.

VI. FUTURE WORK

There are several ways this could be extended. In this project we have tested class weighting. The natural next step is to compare it with oversampling methods like SMOTE [24] to see which approach handles the imbalance best. The decision threshold could also be tuned so that the bank can choose for itself the balance between precision and recall depending on the cost of a missed client. The models can be improved further using cross-validation with proper hyperparameter tuning. Finally, other algorithms such as SVM or boosting methods could be added to the comparison.

REFERENCES

[1] S. Moro, P. Cortez, and P. Rita, "A data-driven approach to predict the success of bank telemarketing," Decision Support Systems, vol. 62, pp. 22–31, 2014.

[2] J. Han, M. Kamber, and J. Pei, Data Mining: Concepts and Techniques, 3rd ed. Waltham, MA: Morgan Kaufmann, 2011.

[3] P.-N. Tan, M. Steinbach, A. Karpatne, and V. Kumar, Introduction to Data Mining, 2nd ed. New York: Pearson, 2019.

[4] D. Dua and C. Graff, UCI Machine Learning Repository. Irvine, CA: University of California, School of Information and Computer Science, 2019. [Online]. Available: https://archive.ics.uci.edu/ml

[5] S. Moro, R. Laureano, and P. Cortez, "Using data mining for bank direct marketing: An application of the CRISP-DM methodology," in Proc. European Simulation and Modelling Conf. (ESM), 2011, pp. 117–121.

[6] H. A. Elsalamony, "Bank direct marketing analysis of data mining techniques," International Journal of Computer Applications, vol. 85, no. 7, pp. 12–22, 2014.

[7] C. B. Latha and S. C. Jeeva, "Improving the accuracy of prediction of heart disease risk based on ensemble classification techniques," Informatics in Medicine Unlocked, vol. 16, 100203, 2019.

[8] L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone, Classification and Regression Trees. Belmont, CA: Wadsworth, 1984.

[9] J. R. Quinlan, "Induction of decision trees," Machine Learning, vol. 1, no. 1, pp. 81–106, 1986.

[10] T. Bayes, "An essay towards solving a problem in the doctrine of chances," Philosophical Transactions of the Royal Society of London, vol. 53, pp. 370–418, 1763.

[11] P. Domingos and M. Pazzani, "On the optimality of the simple Bayesian classifier under zero-one loss," Machine Learning, vol. 29, no. 2–3, pp. 103–130, 1997.

[12] T. Cover and P. Hart, "Nearest neighbor pattern classification," IEEE Transactions on Information Theory, vol. 13, no. 1, pp. 21–27, 1967.

[13] L. Breiman, "Random forests," Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.

[14] T. K. Ho, "Random decision forests," in Proc. 3rd Int. Conf. Document Analysis and Recognition (ICDAR), 1995, pp. 278–282.

[15] D. R. Cox, "The regression analysis of binary sequences," Journal of the Royal Statistical Society: Series B, vol. 20, no. 2, pp. 215–242, 1958.

[16] J. MacQueen, "Some methods for classification and analysis of multivariate observations," in Proc. 5th Berkeley Symp. Mathematical Statistics and Probability, vol. 1, 1967, pp. 281–297.

[17] S. Lloyd, "Least squares quantization in PCM," IEEE Transactions on Information Theory, vol. 28, no. 2, pp. 129–137, 1982.

[18] I. T. Jolliffe, Principal Component Analysis, 2nd ed. New York: Springer, 2002.

[19] T. Fawcett, "An introduction to ROC analysis," Pattern Recognition Letters, vol. 27, no. 8, pp. 861–874, 2006.

[20] J. Davis and M. Goadrich, "The relationship between precision-recall and ROC curves," in Proc. 23rd Int. Conf. Machine Learning (ICML), 2006, pp. 233–240.

[21] D. M. W. Powers, "Evaluation: From precision, recall and F-measure to ROC, informedness, markedness and correlation," Journal of Machine Learning Technologies, vol. 2, no. 1, pp. 37–63, 2011.

[22] H. He and E. A. Garcia, "Learning from imbalanced data," IEEE Transactions on Knowledge and Data Engineering, vol. 21, no. 9, pp. 1263–1284, 2009.

[23] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," PLoS ONE, vol. 10, no. 3, e0118432, 2015.

[24] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," Journal of Artificial Intelligence Research, vol. 16, pp. 321–357, 2002.

[25] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[26] C. R. Harris et al., "Array programming with NumPy," Nature, vol. 585, pp. 357–362, 2020.

[27] W. McKinney, "Data structures for statistical computing in Python," in Proc. 9th Python in Science Conf. (SciPy), 2010, pp. 56–61.

[28] J. D. Hunter, "Matplotlib: A 2D graphics environment," Computing in Science & Engineering, vol. 9, no. 3, pp. 90–95, 2007.

[29] M. L. Waskom, "Seaborn: Statistical data visualization," Journal of Open Source Software, vol. 6, no. 60, 3021, 2021.

[30] T. Hastie, R. Tibshirani, and J. Friedman, The Elements of Statistical Learning: Data Mining, Inference, and Prediction, 2nd ed. New York: Springer, 2009.

[31] C. M. Bishop, Pattern Recognition and Machine Learning. New York: Springer, 2006.

[32] G. James, D. Witten, T. Hastie, and R. Tibshirani, An Introduction to Statistical Learning. New York: Springer, 2013.

[33] C. C. Aggarwal, Data Mining: The Textbook. Cham, Switzerland: Springer, 2015.

[34] P. J. Rousseeuw, "Silhouettes: A graphical aid to the interpretation and validation of cluster analysis," Journal of Computational and Applied Mathematics, vol. 20, pp. 53–65, 1987.